Controlling Transactions and Locks Part 4 : SQL Snapshot

By Don Schlichting

Snapshot

Unlike table hints, where the standard locks and methods that have been discussed so far are applied to a transaction at run time, without any further configuration, Snapshots rely on an entirely new data change tracking method, configured at the database level. This new method is more than just a slight logical change; it requires the server to handle the data physically different from before. Once this new data change tracking method is enabled, it creates a copy, or snapshot of every data change. By reading these snapshots rather than live data at times of contention, Shared Locks are no longer needed on reads, and overall database performance may increase. This new change tracking method is called Row Versioning.

Row Versioning

Although Row Versioning is new to Microsoft SQL Server, it exists in the Oracle Database products. For Microsoft SQL users (and most other database users other than Oracle), the database is handled in a Pessimistic way. The database philosophically expects there will be many data conflicts; with multiple sessions all trying to change the same data at the same time and corruption will result. To avoid this, Locks are put in place to guard data integrity. As we have examined in the previous articles, even the simplest read produces a Shared Lock. For databases with heavy insert, update, and delete activity, this is an excellent methodology. Locking safeguards against data corruption, but that safety comes at the price of requiring system resources for lock escalations and management. The locking also produces the side effect of Readers blocking Writers overviewed in the previous article. There are a few instances though, when this pessimistic heavy lock design is more of a negative than a positive benefit, such as applications that have very heavy read activity with light writes, or when porting applications from Oracle to MS SQL. For applications with heavy read activity, the first option might be to check the viability of using the NOLOCK hint. NOLOCK will not issue a Shared Lock and will not honor locks issued by other transactions. As a result, performance may increase. Unfortunately, you are never sure about the integrity of the data being returned by your NOLOCK statement, it may be either committed or uncommitted data. If this is unacceptable, Row Versioning may be a good fit. Row Versioning may also be a good for fit for off the shelf or third party applications, where it is not practical to change hundreds of stored procedures to use lock hints, only to have your changes undone during the next upgrade. Additionally, if their database statements are inside complied code, rather than database procedures, then without source code, applying lock hints is not an option. Fortunately, Row Versioning is enabled at the database level with one statement, and once enabled, no additional changes need to be made at the statement or procedure level.

Row Versioning works by writing a copy of any data about to be changed to a Version Store, located in the system database tempdb. This copy is the value of the data prior to the change. If the changing transaction rolls back, then the data is copied out of the version store and back into the live database. If committed, the row in the Version Store is deleted. For example, with Row Versioning enabled, an update with Commit would look like this:

Transaction 1 will change an employee name from 'Kim' to 'Jane';
Kim is written to the Version Store;
Jane is written to the database;
Commit
Jane in the database is left as is;
Kim in the Version Store is deleted;

An update with a Roll Back would do this:

Transaction 1 will change an employee name from 'Kim' to 'Jane';
Kim is written to the Version Store;
Jane is written to the database;
Roll Back
Jane is overwritten with Kim from the Version Store;
Kim in the Version Store is deleted;

Using the same example, we will include a second transaction trying to read to same data the first transaction is modifying. Without Row Versioning, the second transaction would be blocked, sitting idle, until the first transaction either committed or rolled back. However, with Row Versioning, the Data Store is used to speed up this process.

Transaction 1 will change an employee name from 'Kim' to 'Jane';
Kim is written to the Version Store;
Jane is written to the database;
Transaction 2 tries to read the same row; Tran 1 has not finished yet;
Tran 2 looks at the row in the live database, sees a pointer to the version store;
Looks in Version Store for the most recently committed data, in this case "Kim";
"Kim" is returned to the read statement;

Therefore, with Row Versioning, we are getting the benefit of a committed read with out a block. In our example, the last committed read was "Kim." If three transactions change the same row, then there will be three rows in the Version Store. During a read, the Version Store will be checked for the most current committed data during the time the read was issued. In our examples, Rows in the Version Store were deleted on Commit or Roll Back, but in reality, a garbage collector periodically deletes them. Notice the extra work required by writes. Every DML requires the extra task or writing to the version store, regardless if there are active readers or not. If there are no active transactions needing the data, then the garbage collector will periodically delete the version store rows.

Read Committed Snapshot

Our examples demonstrate a version of Snapshot called Read Committed. To enable it, execute the following statement,

```
ALTER DATABASE adventureworks
SET READ_COMMITTED_SNAPSHOT ON
GO
```

Snapshot is now enabled for the database. To disable, use,

```
ALTER DATABASE adventureworks
```

```
SET READ_COMMITTED_SNAPSHOT OFF
GO
```

Conclusion

Snapshots in SQL provide new locking options. Read Committed Snapshot is very simple to enable and disable, so testing applications for performance gains should be straightforward. Next month, we will explore Snapshot deeper and look at an optional isolation level.